

Novelty Library WebApp — Engineering & Product Rules

These rules are derived from the current implementation and should govern future changes unless a deliberate architecture/product decision supersedes them.

1. Source-of-truth rules

1. Treat the codebase as the implementation source of truth for current behavior.
2. Treat Supabase/RPC authorization as the security source of truth.
3. Treat `src/lib` service functions as the preferred application data-access layer.
4. Preserve built-in defaults whenever database-driven configuration is absent or invalid.

2. Supabase rules

1. **Use one Supabase client only.** Import `supabase` from `src/lib/supabase.ts`.
2. Never call `createClient()` from another feature module.
3. `communitySupabase.ts` must continue to refer to the same client unless the architecture is explicitly redesigned.
4. Never put a service-role key in frontend code.
5. Client-side admin checks are for UX only; database policies/RPCs must enforce authorization.
6. Handle database errors explicitly on user-facing mutations.
7. Non-critical integrations may degrade without blocking the primary reading experience.

3. Authentication rules

1. Auth state must be managed through `AuthProvider/useAuth`.
2. Do not maintain a second independent auth state machine.
3. Email addresses should be normalized to lowercase on authentication operations.
4. Google OAuth redirect URLs must use the current site origin/base URL.
5. Password-recovery and expired-link states must remain understandable to users.
6. Signing out must clear local application `auth/profile/admin` state.

4. Routing rules

1. Preserve existing hash-route compatibility.
2. Route parsing belongs in `src/lib/router.ts`.
3. Pages should receive route parameters rather than parse the hash independently unless an action fragment explicitly requires it.
4. If a new route is introduced, add it to the typed `Route` union and `AppContent`.
5. Do not silently break deep links such as review slugs and public usernames.

5. Review rules

1. Review status values are pending, approved, declined.
2. A declined review must never render publicly.
3. Publication must be explicit.
4. Reviewer R/W Rating is the reviewer's own opinion.
5. NL Rating is the cumulative average of reader ratings on the published community review post.
6. Never include R/W Rating in the NL community average.
7. Preserve reviewer attribution wherever the review is displayed.
8. Preserve external poster/purchase links only after URL validation.
9. Book duplicate/reservation checks should happen before accepting a new review submission where applicable.

6. Submission rules

1. Required submission fields must remain clearly marked.
2. Review text must meet the current minimum quality/length validation.
3. The undertaking checkbox is mandatory for submission completion.
4. The honeypot field must remain invisible to ordinary users.
5. Drafts must not contain the honeypot value.
6. Post-submit editing is time-limited; do not imply indefinite editing.
7. A refresh after submission should not accidentally reopen the editing window.
8. Cover sources must be tracked consistently.

7. Text/content safety rules

1. Sanitize user-generated text before rendering.
2. Validate external URLs before using them as links.
3. Do not render arbitrary HTML from user fields without sanitization.
4. Preserve line breaks/paragraphs where editorial readability depends on them.
5. Keep user-facing errors understandable and non-technical.

8. Profile rules

1. Name and account email are locked core profile fields in admin configuration.
2. Built-in profile fields and dynamic profile questions are separate concepts.
3. Dynamic answers are keyed by stable question keys.
4. Profile question keys must remain stable after publication; changing a key can orphan existing answers.
5. Multi-select limits must be enforced both in the UI and ideally in backend validation/RLS/database logic where feasible.
6. Image limits must be enforced before upload and should be validated again server-side where feasible.
7. Profile visibility must be respected on public profiles.
8. Follower/following privacy settings must not leak hidden connections.

9. Profile-card rules

1. The preview is intended to represent the export canvas.
2. Export dimensions must match the selected format.
3. The export should not contain entrance animations or transient UI state.
4. Overflow handling must be deterministic.
5. Never allow a long name/title/question to visually escape the canvas.
6. Keep the 4:5 and 16:9 special compositions unless a new design replaces them deliberately.
7. Dynamic profile questions must respect their configured card mode.
8. Tags/answers must be capped intentionally; do not render unlimited user selections into a fixed social card.

10. Community engagement rules

1. A user can have at most one feedback record per review post according to the current `review_id, user_id` upsert model.
2. Likes are user-specific.
3. Optimistic like updates must roll back on failure.
4. Engagement-sheet synchronization is best-effort and must not block the UI.
5. Community review content must be sanitized.
6. Community ratings must remain independent from the original reviewer rating.

11. Social/story rules

1. Stories expire after their configured 24-hour lifetime.
2. Expired stories must not be presented as active.
3. Story media belongs in the stories storage bucket.
4. Reposts/follows must use backend RPCs where the current implementation does so.
5. User privacy controls override convenience in connection displays.

12. Admin rules

1. Admin pages require authenticated admin status.
2. The UI must provide an access-denied state rather than rendering privileged controls.
3. Destructive admin actions require clear confirmation.
4. Admin configuration must have defaults/fallbacks.
5. Public-facing copy edits should not require a redeploy.
6. Finance links are configuration, not payment processing logic.
7. The Supabase Dashboard shortcut must not expose credentials.

13. Content management rules

1. Editable page content is stored through the configured `editable_pages` mechanism.
2. About content uses structured sections rather than one giant HTML blob.

3. Review guidelines remain editable and version-safe.
4. If DB content is empty or malformed, render safe built-in defaults.

14. Analytics rules

1. Public analytics must be aggregate-only.
2. Never expose private emails or private account metadata through `home_analytics()`.
3. Analytics animations must respect reduced-motion preferences.
4. Avoid misleading milestone/counter presentation; label approximations and + notation clearly.

15. UI rules

1. Use the existing teal/cyan token system.
2. Use Fraunces/serif for editorial headings and Inter/system sans for UI text.
3. Preserve light/dark themes.
4. Prefer rounded surfaces and restrained shadows.
5. Avoid adding arbitrary colors that break the Novelty Library palette.
6. Do not sacrifice readability for visual effects.
7. New components should reuse existing button/input/surface patterns.

16. Accessibility rules

1. Interactive controls require accessible names.
2. Keyboard users must be able to complete core flows.
3. Do not use color alone to indicate status.
4. Respect reduced motion.
5. Maintain adequate contrast.
6. Use meaningful alternative text for editorial imagery.

17. Dependency rules

Before adding a dependency:

- Confirm the requirement cannot reasonably be met with the current stack.
- Check bundle/runtime cost.
- Prefer maintained libraries.
- Avoid duplicate functionality already covered by React, Tailwind, Lucide, DOMPurify, Supabase or existing utilities.

18. Change-management rules

Every substantial feature should update:

- `PRD.md` if product behavior changes.
- `ARCHITECTURE.md` if data/technical architecture changes.
- `DESIGN.md` if visual behavior changes.

- RULES.md if a new invariant is introduced.
- TASKS.md with implementation status.
- README.md if setup/operation changes.

19. Quality gates

Before deployment, run:

```
npm run typecheck
npm run lint
npm run build
```

A change is not production-ready if any of these fail unless the failure is documented and intentionally accepted.

20. Built-in profile field placement

1. Built-in field placement is stored as an optional section override in `profile-core-fields` (only when it differs from the default) plus the shared order map in `profile-core-field-order`.
2. Name and Email are locked: they can never be moved, hidden or relabelled.
3. A built-in field's answer type is fixed (it maps to a typed `profiles` column); only label, placeholder, section, visibility defaults and active state are editable.
4. Any built-in section save must persist every built-in field value, because fields can be moved between sections.
5. Do not place a full-width `<button>` inside a draggable element.

21. Converted reading questions

1. Questions whose key is one of `reading_since`, `books_read_this_month`, `total_books_read`, `favorite_book`, `favorite_author`, `favorite_genre` are bridged to the `profiles` column of the same name, not to `profile_answers`. Never change these keys.
2. The profile card shows these six through its fixed `stat/rows` layout; they must be excluded from the dynamic question list to avoid duplicates.
3. Total Books Read is still floored at the reader's accepted-review count on save.
4. The conversion is done by `convertReadingJourneyToQuestions()` (`src/lib/profileReadingMigration.ts`) and marks `reading_journey.converted` in `profile-section-layout`; it must stay idempotent.